IDAT7215
Computer Programming for Product Development and Applications

Lecture 2-4: Python Libraires: Pandas

Dr. Zulfiqar Ali

Outline

- Data Manipulation
- DataFrame
- Statistics

Pandas

Pandas

- In the NumPy section we dealt with some arrays, whose columns had each a special meaning. For example, the column number 0 could contain values interpreted as years, and column 1 could contain a month, and so on.

- It is possible to handle the data this way, but in can be hard to remember, which column number corresponds to which variable. Especially, if you later remove some column from the array, then the numbering of the remaining columns changes.

Pandas

- One solution to this is to give a descriptive name to each column. These column names stay fixed and attached to their corresponding columns, even if we remove some of the columns.

- In addition, the rows can be given names as well, these are called indices in Pandas.

Pandas

- The Pandas library is built on top of the NumPy library, and it provides a special kind of two-dimensional data structure called `DataFrame`.

- The `DataFrame` allows to give names to the columns, so that one can access a column using its name in place of the index of the column.

What is Data?Iris Data set

- Many data analytics techniques are illustrated with the Iris Plant data set.
  - Details of this data set can be found https://en.wikipedia.org/wiki/Iris_flower_data_set
  - Three flower types (classes):
    - Setosa
    - Versicolor
    - Virginica



Iris-setosa

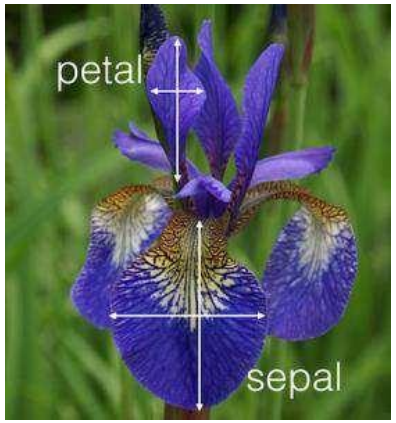


Iris-versicolor



Iris-virginica

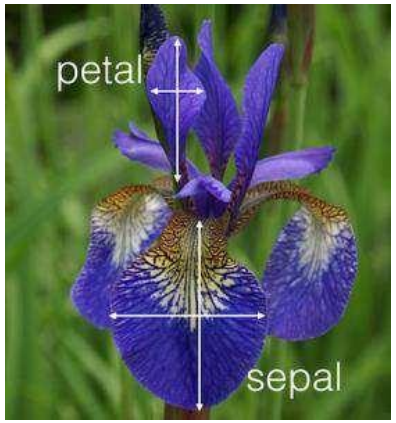
What is Data? Iris Data set



sepal_length	sepal_width	petal_length	petal_width	species
5.1	3.5	1.4	0.2	setosa
4.9	3	1.4	0.2	setosa
7	3.2	4.7	1.4	versicolor
6.3	3.3	6	2.5	virginica
…	…	…	…	…

What is Data? Iris Data set

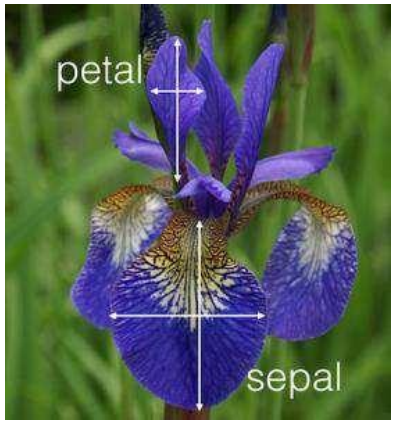
Each row represents one **data sample** (i.e., data point, data object). It represents an entity in real world (e.g., flower).



sepal_length	sepal_width	petal_length	petal_width	species
5.1	3.5	1.4	0.2	setosa
4.9	3	1.4	0.2	setosa
7	3.2	4.7	1.4	versicolor
6.3	3.3	6	2.5	virginica
…	…	…	…	…

What is Data? Iris Data set

Each row represents one **data sample** (i.e., data point, data object). It represents an entity in real world (e.g., flower).

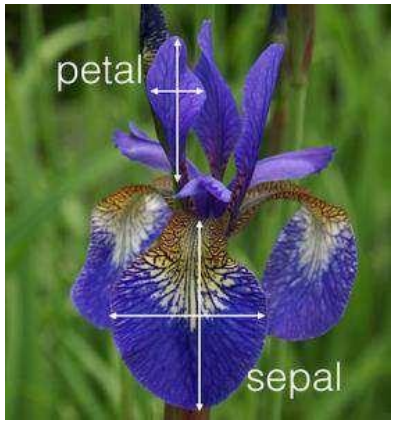


Each column is a **feature** (i.e., **attribute**, **variable**) to describe a characteristic of a data sample.

sepal_length	sepal_width	petal_length	petal_width	species
5.1	3.5	1.4	0.2	setosa
4.9	3	1.4	0.2	setosa
7	3.2	4.7	1.4	versicolor
6.3	3.3	6	2.5	virginica
…	…	…	…	…

What is Data? Iris Data set

Each row represents one **data sample** (i.e., data point, data object). It represents an entity in real world (e.g., flower).



Each column is a **feature** (i.e., **attribute**, **variable**) to describe a characteristic of a data sample.

sepal_length	sepal_width	petal_length	petal_width	species
5.1	3.5	1.4	0.2	setosa
4.9	3	1.4	0.2	setosa
7	3.2	4.7	1.4	versicolor
6.3	3.3	6	2.5	virginica
…	…	…	…	…

- **Data set** is a collections of data samples
- This iris data set contains 150 samples (50 samples from each of three species)
- 5 features: sepal length, sepal width, petal length, petal width, species

Pandas Example

- Import ‘iris.csv’
- We see that the DataFrame contains five columns, four of which are numerical variables.
- We can refer to a column by its name

```
import pandas as pd
df =  pd.read_csv("iris.csv");
df.head(10)
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
5	5.4	3.9	1.7	0.4	setosa
6	4.6	3.4	1.4	0.3	setosa
7	5.0	3.4	1.5	0.2	setosa
8	4.4	2.9	1.4	0.2	setosa
9	4.9	3.1	1.5	0.1	setosa

```
df['sepal_length'].head()
```

```
0    5.1
1    4.9
2    4.7
3    4.6
4    5.0
Name: sepal_length, dtype: float64
```

Summary statistic methods on Pandas columns

- There are several summary statistic methods that operate on a column or on all the columns.

```
print(df['sepal_length'].min())
print(df['sepal_length'].max())
print(df['sepal_length'].mean())
print(df['sepal_length'].median())
```

```
4.3
7.9
5.843333333333335
5.8
```

Drop a column

- We can drop some columns from the `DataFrame` with the `drop` method.
- We can use the `inplace` parameter of the `drop` method to modify the original `DataFrame`.
- Many of the modifying methods of the DataFrame have the `inplace` parameter.

```
df2 = df.drop(["sepal_length","sepal_width"],axis = 1)
df2.head()
```

	petal_length	petal_width	species
0	1.4	0.2	setosa
1	1.4	0.2	setosa
2	1.3	0.2	setosa
3	1.5	0.2	setosa
4	1.4	0.2	setosa

```
df.drop(["sepal_length","sepal_width"],axis = 1, inplace = True)
df.head()
```

	petal_length	petal_width	species
0	1.4	0.2	setosa
1	1.4	0.2	setosa
2	1.3	0.2	setosa
3	1.5	0.2	setosa
4	1.4	0.2	setosa

Add a new column in DataFrame

```
df =  pd.read_csv("iris.csv");
print(df.head(5))
df['label'] = df['species'] == 'setosa'
print(df.head(5))
```

```
   sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa
3           4.6          3.1           1.5          0.2  setosa
4           5.0          3.6           1.4          0.2  setosa
   sepal_length  sepal_width  petal_length  petal_width species  label
0           5.1          3.5           1.4          0.2  setosa   True
1           4.9          3.0           1.4          0.2  setosa   True
2           4.7          3.2           1.3          0.2  setosa   True
3           4.6          3.1           1.5          0.2  setosa   True
4           5.0          3.6           1.4          0.2  setosa   True
```

Creation and indexing of series

- Series is one-dimensional version of DataFrame. Each column in a DataFrame is a Series
- One can turn any one-dimensional list into a Series, which is a one-dimensional data structure.
- We can also attach a name to this series

```
s=pd.Series([1, 4, 5, 2, 5, 2])
s
```

```
0    1
1    4
2    5
3    2
4    5
5    2
dtype: int64
```

```
s.name = "Grades"
s
```

```
0    1
1    4
2    5
3    2
4    5
5    2
Name: Grades, dtype: int64
```

Row indices of Series

- In addition to the values of the series, also the row indices were printed. All the accessing methods from NumPy arrays also work for the Series: indexing, slicing, masking and fancy indexing.

- Note that the indices stick to the corresponding values, they are not renumbered!

```
s=pd.Series([1, 4, 5, 2, 5, 2])
print(s[1])
s2=s[[0,5]]
print(s2)
```

```
4
0    1
5    2
dtype: int64
```

```
print(s2[5])
print(s2[1])
```

```
2
---------------------------------------
KeyError
```

Series

- The values as a NumPy array are accessible via the `values` attribute.
- The `indices` are available through the index attribute.
- The index is not simply a NumPy array, but a data structure that allows fast access to the elements.

```
s=pd.Series([1, 4, 5, 2, 5, 2])
s2=s[[0,5]]
print(s2.values)
print(s2.index)
```

```
[1 2]
Int64Index([0, 5], dtype='int64')
```

```
s3=pd.Series([1, 4, 5, 2, 5, 2], index=list("abcdef"))
s3
```

```
a    1
b    4
c    5
d    2
e    5
f    2
dtype: int64
```

```
print(s3.index)
print(s3['c'])
```

```
Index(['a', 'b', 'c', 'd', 'e', 'f'], dtype='object')
5
```

Series

- It is still possible to access the series using NumPy style *implicit integer indices.*
- This can be confusing though.
- Pandas offers attributes `loc` and `iloc`. The attributes `loc` always uses the explicit index, while the attribute `iloc` always uses the implicit integer index

```
print(s3)
print(s3[0])
```

```
a    1
b    4
c    5
d    2
e    5
f    2
dtype: int64
1
```

```
s4 = pd.Series(["Jack", "Jones", "James"], index=[1,2,3])
s4
```

```
1     Jack
2    Jones
3    James
dtype: object
```

```
s4[1]
```

```
'Jack'
```

```
s4.iloc[1]
```

```
'Jones'
```

```
s4.loc[1]
```

```
'Jack'
```

Creation of dataframes

- The DataFrame is essentially a two-dimensional object, and it can be created in three different ways:
  - out of a two-dimensional NumPy array
  - out of given columns
  - out of given rows

Creating DataFrames from a NumPy array

- In the following example a DataFrame with 2 rows and 3 column is created. The row and column indices are given explicitly.

```
# DataFrames
df=pd.DataFrame(np.random.randn(2,3), columns=["First", "Second", "Third"], index=["a", "b"])
df
```

	First	Second	Third
a	0.149066	-0.176563	-1.590523
b	-0.015790	1.329185	0.420429

- If either columns or index argument is left out, then an implicit integer index will be used.

```
df2=pd.DataFrame(np.random.randn(2,3), index=["a", "b"])
df2
```

	0	1	2
a	1.210334	-1.035517	0.494750
b	-0.076913	-0.655023	1.235455

Creating DataFrames from columns

- A column can be specified as a list, an NumPy array, or a Pandas' Series.

- Input is a dictionary of which keys give the column names and values are the actual column content.

```
s1 = pd.Series([1,2,3])
s2 = pd.Series([4,5,6])
df = pd.DataFrame({'a':s1, 'b':s2})
print(df)
```

```
   a  b
0  1  4
1  2  5
2  3  6
```

Creating DataFrames from rows

- We can give a list of rows as a parameter to the DataFrame constructor.

```
df=pd.DataFrame([{"Wage" : 1000, "Name" : "Jack", "Age" : 21}, {"Wage" : 1500, "Name" : "John", "Age" : 29}])
df
```

	Age	Name	Wage
0	21	Jack	1000
1	29	John	1500

```
df = pd.DataFrame([[1000, "Jack", 21], [1500, "John", 29]], columns=["Wage", "Name", "Age"])
df
```

	Wage	Name	Age
0	1000	Jack	21
1	1500	John	29

Accessing columns and rows of a dataframe

- Recommend to use attributes `loc` and `iloc` for accessing columns and rows in a dataframe.
- `loc` uses explicit indices and the `iloc` uses the implicit integer indices.

```
df = pd.DataFrame([[1000, "Jack", 21], [1500, "John", 29]], columns=["Wage", "Name", "Age"])
df
```

	Wage	Name	Age
0	1000	Jack	21
1	1500	John	29

```
print(df.loc[1, "Wage"])

print(df.iloc[0,0])

print(df.loc[1, ["Name", "Wage"]])
```

```
1500
1000
Name    John
Wage    1500
Name: 1, dtype: object
```

Accessing columns and rows of a dataframe

- With `iloc` everything works like with NumPy arrays: indexing, slicing, fancy indexing, masking and their combinations.

- With `loc` it is the same but now the names in the explicit indices are used for specifying rows and columns.

Summary statistics of Pandas

- The summary statistic methods work in a similar way as their counter parts in NumPy. By default, the aggregation is done over columns.

- The `describe` method of the DataFrame object gives different summary statistics for each (numeric) column. The result is a DataFrame. This method gives a good overview of the data and is typically used in the exploratory data analysis phase.

```
df =  pd.read_csv("iris.csv");
df.describe()
```

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.054000	3.758667	1.198667
std	0.828066	0.433594	1.764420	0.763161
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

Use Summary Statistics to Explore Data

Use Summary Statistics to Explore Data (in Python)

In [3]:
```
# Libraries
import pandas as pd

# load data
iris = pd.read_csv("iris.csv")

iris.head()
```

Out[3]:

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

- Load and see the data

Feature Type

```
iris.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
sepal_length    150 non-null float64
sepal_width     150 non-null float64
petal_length    150 non-null float64
petal_width     150 non-null float64
species         150 non-null object
dtypes: float64(4), object(1)
memory usage: 5.9+ KB
```

- Show the feature types

Summary statistic in python

```
In [6]: iris.describe()
```

Out[6]:

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.054000	3.758667	1.198667
std	0.828066	0.433594	1.764420	0.763161
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

- Mean
- Standard Deviation
- Min
- Q1
- Q2
- Q3
- Max

Summary statistic in python: mean; standard deviation; median

```
In [7]: iris['sepal_length'].std()
Out[7]: 0.8280661279778629

In [9]: iris['sepal_length'].mean()
Out[9]: 5.843333333333335

In [10]: iris['sepal_length'].median()
Out[10]: 5.8
```

- Compute the mean, standard deviation and median for the feature ‘sepal_length’

Summary statistic in python: Mode

```
In [13]: iris['species'].value_counts()

Out[13]: setosa        50
         versicolor    50
         virginica     50
         Name: species, dtype: int64
```

```
In [14]: iris['species'].mode()

Out[14]: 0        setosa
         1    versicolor
         2     virginica
         dtype: object
```

- Get counts of unique values for ‘species’
- Compute the mode for ‘species’

Summary statistics for different groups

- Explore more about the data

Setosa

```
In [17]: iris_setosa = iris[iris['species'] == 'setosa']
         iris_setosa.describe()
```

Out[17]:

	sepal_length	sepal_width	petal_length	petal_width
count	50.00000	50.000000	50.000000	50.00000
mean	5.00600	3.418000	1.464000	0.24400
std	0.35249	0.381024	0.173511	0.10721
min	4.30000	2.300000	1.000000	0.10000
25%	4.80000	3.125000	1.400000	0.20000
50%	5.00000	3.400000	1.500000	0.20000
75%	5.20000	3.675000	1.575000	0.30000
max	5.80000	4.400000	1.900000	0.60000

Versicolor

```
In [18]: iris_versicolor = iris[iris['species'] == 'versicolor']
         iris_versicolor.describe()
```

Out[18]:

	sepal_length	sepal_width	petal_length	petal_width
count	50.000000	50.000000	50.000000	50.000000
mean	5.936000	2.770000	4.260000	1.326000
std	0.516171	0.313798	0.469911	0.197753
min	4.900000	2.000000	3.000000	1.000000
25%	5.600000	2.525000	4.000000	1.200000
50%	5.900000	2.800000	4.350000	1.300000
75%	6.300000	3.000000	4.600000	1.500000
max	7.000000	3.400000	5.100000	1.800000

The petal_length of ‘setosa’ is always smaller than the petal_length of ‘versicolor’

Summary statistics for different group

- Explore more about the data

Setosa

```
In [17]: iris_setosa = iris[iris['species'] == 'setosa']
         iris_setosa.describe()
```

Out[17]:

	sepal_length	sepal_width	petal_length	petal_width
count	50.00000	50.000000	50.000000	50.00000
mean	5.00600	3.418000	1.464000	0.24400
std	0.35249	0.381024	0.173511	0.10721
min	4.30000	2.300000	1.000000	0.10000
25%	4.80000	3.125000	1.400000	0.20000
50%	5.00000	3.400000	1.500000	0.20000
75%	5.20000	3.675000	1.575000	0.30000
max	5.80000	4.400000	1.900000	0.60000

Versicolor

```
In [18]: iris_versicolor = iris[iris['species'] == 'versicolor']
         iris_versicolor.describe()
```

Out[18]:

	sepal_length	sepal_width	petal_length	petal_width
count	50.000000	50.000000	50.000000	50.000000
mean	5.936000	2.770000	4.260000	1.326000
std	0.516171	0.313798	0.469911	0.197753
min	4.900000	2.000000	3.000000	1.000000
25%	5.600000	2.525000	4.000000	1.200000
50%	5.900000	2.800000	4.350000	1.300000
75%	6.300000	3.000000	4.600000	1.500000
max	7.000000	3.400000	5.100000	1.800000

The petal_width of ‘setosa’ is always smaller than the petal_width of ‘versicolor’

Summary statistics for different group

- Explore more about the data

Setosa

```
In [17]: iris_setosa = iris[iris['species'] == 'setosa']
         iris_setosa.describe()
```

Out[17]:

	sepal_length	sepal_width	petal_length	petal_width
count	50.00000	50.000000	50.000000	50.00000
mean	5.00600	3.418000	1.464000	0.24400
std	0.35249	0.381024	0.173511	0.10721
min	4.30000	2.300000	1.000000	0.10000
25%	4.80000	3.125000	1.400000	0.20000
50%	5.00000	3.400000	1.500000	0.20000
75%	5.20000	3.675000	1.575000	0.30000
max	5.80000	4.400000	1.900000	0.60000

Versicolor

```
In [18]: iris_versicolor = iris[iris['species'] == 'versicolor']
         iris_versicolor.describe()
```

Out[18]:

	sepal_length	sepal_width	petal_length	petal_width
count	50.000000	50.000000	50.000000	50.000000
mean	5.936000	2.770000	4.260000	1.326000
std	0.516171	0.313798	0.469911	0.197753
min	4.900000	2.000000	3.000000	1.000000
25%	5.600000	2.525000	4.000000	1.200000
50%	5.900000	2.800000	4.350000	1.300000
75%	6.300000	3.000000	4.600000	1.500000
max	7.000000	3.400000	5.100000	1.800000

- We have our own way to distinguish ‘setosa’ and ‘versicolor’ (i.e., using petal_length or petal_width)
- The knowledge is gained by performing simple summary statistics techniques on data.

The pattern is more obvious when we visualize it

- The petal_length of ‘setosa’ is always smaller than the petal_length of ‘versicolor’
- The petal_width of ‘setosa’ is always smaller than the petal_width of ‘versicolor’
- It is very easy to classify ‘setosa’ and ‘virginica’ just based on petal_length and petal_width.

